ALGORITHM DESIGN CHALLENGE ASSESSMENT

Q1: Greedy Best-First Search (Romania Map)

Problem Statement

The objective is to find a navigational path from a starting city to a destination city on the map of Romania using an algorithm that prioritizes cities that appear to be closest to the goal based on an estimated distance.

Problem Analysis

Initial State	**The starting city on the map (e.g., Arad).**
Goal State	**The destination city (e.g., Bucharest).**
Assumptions	**A valid path exists; straight-line distance coordinates to the destination are known for all cities.**
Constraints	**The agent can only travel along predefined roads between adjacent cities.**

Algorithm Selection & Justification

AI Technique Selection: Greedy Best-First Search.

Justification of Algorithm: Greedy Best-First Search is highly suitable here because it utilizes heuristic information (straight-line distance) to guide the search directly toward the goal. Compared to uninformed searches like Breadth-First Search (which expands blindly in all directions), Greedy search explores far fewer nodes by continually stepping toward the node that appears closest to the target, yielding a rapid solution.

State Space & Heuristics

State Space Representation	**A search graph where nodes represent Romanian cities and edges represent the roads connecting them.**
Variables, Domains, Constraints	**Not Applicable.**
Heuristic Function	**Let *h(n)* be the straight-line distance (SLD) from city *n* to Bucharest. It is appropriate because it provides a geometrically logical estimate of remaining travel, though it does not guarantee optimality.**

Working / Execution Trace

Suppose the start node is Arad and the goal is Bucharest. Arad expands to Sibiu, Timisoara, and Zerind. The algorithm calculates ***h(n)*** for each. Sibiu has the lowest SLD to Bucharest. The algorithm selects Sibiu, expands it, and again picks the adjacent city with the smallest SLD. This continues until Bucharest is reached.

Analysis & Conclusion

Complexity Analysis Time Complexity:

O(b^m) where b is branching factor and m is max depth. Space Complexity: O(b^m) (maintaining frontier and explored sets).

Advantages & Limitations Advantages:

Often much faster than uninformed search strategies. Limitations: It is neither optimal nor complete; it can get trapped in dead ends or loops if cycle detection is absent.

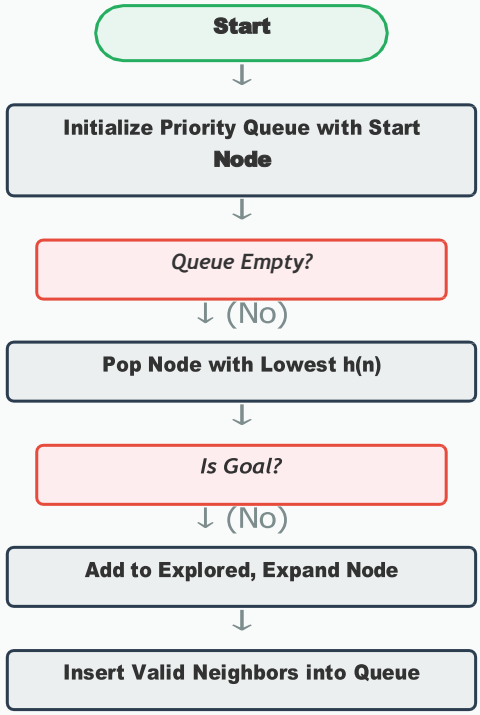
Conclusion:

Greedy Best-First Search is effective for finding a quick, satisficing route on geographical maps. If strict path optimality is required, A* Search should be used instead.

Pseudocode

```
function GREEDY_BFS(problem): node =
    NODE(problem.INITIAL) if
    problem.IS_GOAL(node):
        return SOLUTION(node)

    frontier =
PriorityQueue(order_by="h(n)")
    frontier.insert(node) explored =
    set()

    while not frontier.is_empty(): node =
        frontier.pop()
# Lowest h(n)

        if problem.IS_GOAL(node):
            return SOLUTION(node)

        explored.add(node)

        for child in
problem.EXPAND(node):
            if child not in explored
and frontier:
                frontier.insert(child)
```

Flowchart



Q2: A* Search (Delivery Robot)

Problem Statement

A delivery robot needs to compute the absolute shortest, most cost-effective path from a warehouse to a customer's location traversing a network of weighted road segments.

Problem Analysis

Initial State	The robot's starting position at the warehouse.
Goal State	The customer's specific geographic location.
Assumptions	The environment is static, road weights (costs) are non-negative, and the layout is fully known.
Constraints	The robot cannot traverse non-navigable areas and must minimize the cumulative travel cost.

Algorithm Selection & Justification

AI Technique Selection: A* (A-Star) Search.

Justification of Algorithm: A* is selected over algorithms like Dijkstra's or Greedy BFS because it combines the best of both: it tracks the actual cost from the start ***g(n)*** to ensure optimality, while using a heuristic ***h(n)*** to guide the search efficiently toward the goal. As long as the heuristic is admissible, A* guarantees finding the optimal path while exploring fewer nodes than Dijkstra's.

State Space & Heuristics

State Space Representation	A weighted search graph or grid, where nodes represent intersections and edges represent travel costs.
Variables, Domains, Constraints	Not Applicable.
Heuristic Function	Let ***h(n)*** be the Euclidean or Manhattan distance. It is admissible because a straight line (or unblocked grid path) is always shorter than or equal to the actual road distance.

Working / Execution Trace

Start at warehouse (Node S). Set ***g(S)=0***. Calculate ***f(S) = g(S) + h(S)***. Expand S to neighbors A and B. Calculate ***f(A) = g(A) + h(A)*** and ***f(B) = g(B) + h(B)***. Pick the node with the lowest ***f(n)***. If a shorter path to an already discovered node is found, update its ***g(n)*** and parent pointer. Repeat until the target node is popped from the open list.

Analysis & Conclusion

Complexity Analysis	Time Complexity: *O(b^d)* worst case. Space Complexity: *O(b^d)* because it keeps all generated nodes in memory.
Advantages & Limitations	Advantages: Optimal and complete. Highly efficient pathfinding. Limitations: High memory requirement, which can crash systems on very large maps.
Conclusion	A* is the industry standard for static pathfinding. For severely memory-constrained systems, variants like IDA* (Iterative Deepening A*) could be considered as alternatives.

Pseudocode

```
function A_STAR_SEARCH(start, goal):
    open_list = PriorityQueue("f(n)")
    open_list.insert(start, f=0)
    g_score = {start: 0}

    while not open_list.is_empty():
        curr = open_list.pop()

        if curr == goal:
            return
RECONSTRUCT_PATH(curr)

        for nxt, cost in
GET_NEIGHBORS(curr):
            tentative_g = g_score[curr]
+ cost

            if tentative_g <
g_score.get(nxt, ∞):
                g_score[nxt] =
tentative_g
                f_score = tentative_g
+ HEURISTIC(nxt)
                open_list.insert(nxt,
f=f_score)

    return FAILURE
```

Flowchart

Q3: CSP (Exam Timetable - MRV & Forward Checking)

Problem Statement

A university must construct an examination schedule assigning timeslots to subjects such that no student is booked for two overlapping exams simultaneously.

Problem Analysis

Initial State	An empty timetable with unassigned subjects.
Goal State	A complete assignment where every subject is mapped to a timeslot.
Assumptions	A fixed number of exam timeslots/days is available.
Constraints	Conflicting subjects (shared students/professors) cannot share a timeslot.

Algorithm Selection & Justification

AI Technique Selection: Constraint Satisfaction Problem (CSP) formulation using Backtracking Search, augmented with Minimum Remaining Values (MRV) and Forward Checking.

Justification of Algorithm: Standard searching through all permutations of schedules is factorially large and computationally intractable. Formulating this as a CSP allows us to use heuristics like MRV to fail early on bottlenecks, while Forward Checking proactively eliminates invalid future choices, pruning massive branches of the search tree instantly.

State Space & Heuristics

State Space Representation	**A Constraint Graph where nodes are variables (exams) and edges represent hard constraints (conflicts).**
Variables, Domains, Constraints	**Variables:** $X = \{E_1, E_2, ... E_n\}$. **Domains:** $D = \{T_1, T_2, ... T_k\}$. **Constraints:** $E_i eq E_j$ **if they conflict.**
Heuristic Function	**Not Applicable (Uses MRV heuristic, not path-cost** $h(n)$**).**

Working / Execution Trace

Examine all unassigned exams. Select the one with the fewest valid timeslots left (MRV). Assign it to a valid timeslot T_i. Immediately apply Forward Checking: look at all unassigned exams connected to this one, and remove T_i from their domains. If any domain becomes empty, a conflict is inevitable; undo the assignment and backtrack.

Analysis & Conclusion

Complexity Analysis Time:

O(d^n) worst-case, but greatly reduced in practice. Space: O(n imes d) to store domains.

Advantages & Limitations Advantages:

Drastically reduces search time via early pruning. Limitations: Still NP-Hard; extreme cases may still require exponential time.

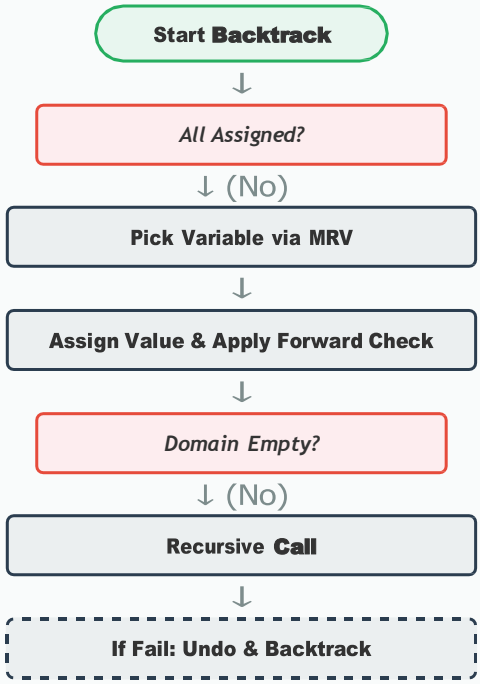
Conclusion:

CSP is the ideal framework for resource allocation and scheduling. Incorporating MRV and Forward Checking makes it practically solvable for large university domains.

Pseudocode

```
function BACKTRACK(assignment, csp):
    if assignment.is_complete():
        return assignment

    var = SELECT_UNASSIGNED(csp,
"MRV")

    for val in ORDER_DOMAIN(var, csp):
        if IS_CONSISTENT(val, var,
csp):
            assignment.add(var = val)

            # Forward Checking
            inferences =
FORWARD_CHK(csp, var)

            if inferences != FAILURE:
                csp.apply(inferences)
                res =
BACKTRACK(assignment, csp)
                if res != FAILURE:
                    return res
                csp.remove(inferences)

            assignment.remove(var) #
Undo

    return FAILURE
```

Flowchart



Q4: Minimax with Alpha-Beta Pruning

Problem Statement

In a zero-sum, adversarial game context, calculate the mathematically optimal move for the MAX player by exploring future game states, while bypassing branches that cannot logically influence the final decision.

Problem Analysis

Initial State	**The current configuration of the game board.**
Goal State	**A winning terminal state or maximum utility score.**
Assumptions	**The opponent (MIN) always plays perfectly to minimize the score.**
Constraints	**Moves must follow strict game rules; turns alternate sequentially.**

Algorithm Selection & Justification

AI Technique Selection: Minimax Algorithm with Alpha-Beta Pruning.

Justification of Algorithm: Minimax provides the theoretical foundation for optimal play in adversarial settings. However, pure Minimax explores every single possibility, which is impossible for games like Chess. Alpha-Beta pruning is applied to identify and eliminate branches where the outcome is already known to be worse than a previously explored path, yielding the exact same result as Minimax but much faster.

State Space & Heuristics

State Space Representation	A Game Tree where the root is the current state, layers alternate between MAX and MIN moves, and leaves are utility values.
Variables, Domains, Constraints	Not Applicable.
Heuristic Function	Not Applicable for pure Minimax (though heuristic evaluation functions are used if depth-limited).

Working / Execution Trace

MAX begins search. It descends leftward to find the utility of the first leaf, say 5. MAX sets ***lpha = 5***. It then explores the next branch. If at an intermediate MIN node, MIN finds a move leading to a utility of 3, MIN will guarantee a score of at most 3. Since MAX already has a guaranteed 5 elsewhere, MAX will never choose this branch. The rest of this MIN branch is pruned.

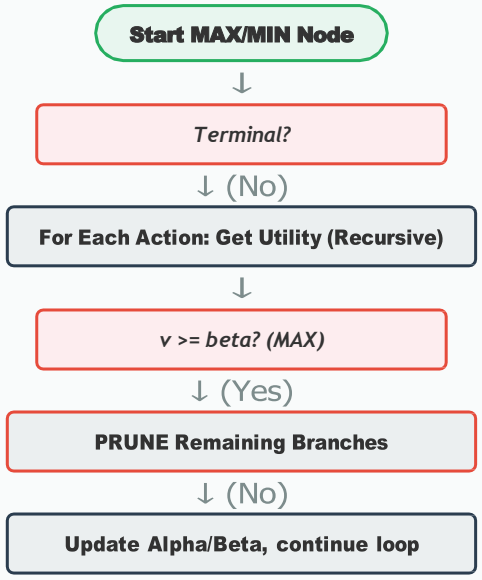
Analysis & Conclusion

Complexity Analysis	**Time: $O(b^{m/2})$ best case (with perfect ordering), $O(b^m)$ worst case. Space: $O(b \times m)$.**
Advantages & Limitations	**Advantages: Returns optimal move; pruning massively expands searchable depth. Limitations: Effectiveness heavily relies on move-ordering heuristics.**
Conclusion	**Alpha-Beta pruning is a fundamental optimization that makes adversarial search viable in complex games, doubling the solvable search depth.**

Pseudocode

```
function MAX_VAL(state, α, β):
    if TERMINAL(state):
        return UTILITY(state)

    v = -∞
    for a in ACTIONS(state):
        v = MAX(v,
MIN_VAL(RESULT(state, a), α, β))

        if v ≥ β:
            return v # Prune

        α = MAX(α, v)

    return v
```

Flowchart



Q5: Online Search Agent (LRTA* Algorithm

Problem Statement

A warehouse robot operates in a partially observable, dynamic environment where obstacles map change without warning. The robot must continuously decide its next immediate move while learning the terrain.

Problem Analysis

Initial State	Robot at start point, with a blank or optimistic map of the environment.
Goal State	Robot at the required destination.
Assumptions	The robot can accurately observe its immediate adjacent squares only.
Constraints	Action execution cannot be reversed; the robot cannot compute a full path offline.

Algorithm Selection & Justification

AI Technique Selection: Online Search (Learning Real-Time A* - LRTA*).

Justification of Algorithm: Offline algorithms like standard A* assume full, static knowledge of the map—they fail completely if an unexpected obstacle blocks the planned route midway. LRTA* interleaves computation and action, allowing the robot to take a step, observe the new state, update its internal heuristic map (learning), and recalculate the next best move on the fly.

State Space & Heuristics

State Space Representation	**A dynamically updated graph based on sensor percepts mapping known states to transitions.**
Variables, Domains, Constraints	**Not Applicable.**
Heuristic Function	**An optimistic estimated cost *H(s)*. When the robot visits a state and finds it blocked or costly, it updates *H(s)* to reflect the real cost, preventing it from getting stuck in dead-end loops.**

Working / Execution Trace

The robot is at state ***S*** and wants to move. It checks neighbors and sees ***A*** and ***B***. It moves to ***A*** based on the lowest heuristic cost. At ***A***, it perceives a wall blocking the path. It updates the internal heuristic

of ***A*** to infinity (or a very high cost). In the next step, avoiding the wall, it moves back to ***S*** and subsequently chooses ***B***, learning the environment incrementally.

Analysis & Conclusion

Complexity Analysis	**Time: *O(1)* computation per step. Space: *O(V)* to store the heuristic table *H(s)*.**
Advantages & Limitations	**Advantages: Functions in unknown/dynamic environments; avoids infinite loops via learning. Limitations: May wander sub-optimally before learning the correct map (exploration penalty).**
Conclusion	**Online search algorithms like LRTA* are indispensable for physical autonomous agents where the environment is unpredictable and real-time execution is required.**

Pseudocode

```
function LRTA_STAR(s_prime):
    if IS_GOAL(s_prime): return STOP

    if IS_NEW(s_prime):
        H[s_prime] = h(s_prime)

    if s is not NULL:
        result[s, a] = s_prime
        H[s] = MIN(c(s,b,next) +
H[next])

    best_a = ARGMIN(c(s_prime,b,next)
+ H[next])

    s = s_prime
    a = best_a

    return a
```

Flowchart